

Senior Data Engineer
Take-Home Technical Challenge

**Analysis of Seasonal Accommodation
Price Trends in Japan**

Tokyo & Osaka · Full Year 2024 · Wanderbricks Dataset

Platform: Databricks Community Edition · Dataset: Wanderbricks
Databricks SQL Dashboard + AI/NL Interface (Genie Space)

1. Project Overview & Business Context

Renters and tourist agencies operating in Japan's accommodation market face a common challenge: without clear visibility into price seasonality, they risk mispricing their offerings or scheduling trips at suboptimal times. This project addresses that gap by building an end-to-end analytical pipeline over the Wanderbricks dataset that surfaces monthly accommodation price trends for Tokyo and Osaka throughout 2024.

The two target audiences are:

- Renters / Property Hosts — need to know when to charge premium prices (high-demand months) and when to adjust rates to remain competitive.
- Tourist Agencies — need to identify low-price windows to package affordable travel itineraries and advise clients accordingly.

The Wanderbricks dataset was selected because it is the native Databricks sample dataset for hospitality, containing realistic booking, property, and destination data that maps directly to this use case. No external data loading is required, making the project fully reproducible from any Databricks Community Edition cluster.

2. Data Architecture

2.1 Architecture Overview

The pipeline follows the Medallion Architecture pattern (Bronze → Silver → Gold), which is typical for Databricks lakehouses. Each layer has a distinct responsibility and progressively increases data quality and analytical value.



2.2 Layer Definitions

BRONZE LAYER — Raw Ingestion	
Source	Wanderbricks dataset on Databricks Community Edition
bookings	booking_id, user_id, property_id, check_in, check_out, guests_count, total_amount, status, created_at, updated_at
properties	property_id, host_id, destination_id, title, description, base_price, property_type, max_guests, bedrooms, bathrooms, property_latitude, property_longitude, created_at
destinations	destination_id, destination, country, state_or_province, state_or_province_code, description
Storage	Delta tables; data read as-is without modification to preserve raw state

❑ SILVER LAYER — Cleaned & Enriched	
Purpose	Clean, join, and filter data to the relevant scope for the analysis
Cleaning	Remove null values in key fields (total_amount, check_in, check_out, destination). Remove exact duplicates. Remove statistical outliers in price.
Joins	bookings ← JOIN → properties (on property_id) ← JOIN → destinations (on destination_id)
Filter	Only bookings in Japan in the year 2024 that had not been canceled
Columns	booking_id, property_id, check_in, check_out, guests_count, total_amount, status, base_price, max_guests, city, property_latitude, property_longitude
Storage	Delta table: silver.japan_bookings_2024

❑ GOLD LAYER — Analytical Aggregates	
Table 1	gold.monthly_price_trends — Aggregated by (city, year, month): avg_price, min_price, max_price, booking_count, total_nights
Table 2	gold.geo_price_index — Aggregated by (property_id, city, year, month): avg_price, min_price, max_price, latitude, longitude — enables map-based views of cheapest areas within each city
Purpose	Pre-aggregated to ensure sub-second query performance on the dashboard without repeated full-table scans

❑ OUTPUT LAYER — Dashboard & AI Interface	
Dashboard	Databricks SQL Dashboard targeting renters and tourist agencies
Chart 1	Average nightly price by month (line chart with each line showing averages for one city)
Chart 2	Monthly booking volume (bar chart showing demand peaks per city)
Chart 3	Combined average spend per month (area chart)
AI Layer	Genie Space (text-to-SQL) over gold.monthly_price_trends for natural-language queries

3. Pipeline Notebooks

3.1 Notebook Structure

All notebooks are designed to run top-to-bottom with a fresh cluster. The repo contains four notebooks:

Notebook	Description
01_bronze	Reads Wanderbricks tables from samples datasets and writes them as Delta tables to the bronze schema. Includes schema inspection and row count validation.
02_silver	Joins and cleans the bronze tables. Applies null removal, deduplication, outlier filtering, city filtering (Tokyo / Osaka), and year filter (2024). Writes to silver.japan_bookings_2024.
03_gold	Aggregates the silver table into two gold tables: monthly_price_trends and geo_price_index. Adds derived columns: total_nights (DATEDIFF), price_per_night (total_amount / total_nights).

3.2 Key Transformations — Bronze Layer (SQL)

```
%sql
CREATE SHCEMA IF NOT EXISTS bronze;
CREATE OR REPLACE TABLE bronze.bookings
AS
SELECT * FROM samples.wanderbricks.bookings;

CREATE OR REPLACE TABLE bronze.properties AS
SELECT * FROM samples.wanderbricks.properties;

CREATE OR REPLACE TABLE bronze.destinations AS
SELECT * FROM samples.wanderbricks.destinations;
```

3.3 Key Transformations — Silver Layer (SQL)

```
%sql
CREATE SCHEMA IF NOT EXISTS silver;

CREATE OR REPLACE TABLE silver.japan_bookings_2024 AS
SELECT
    b.booking_id,
    b.property_id,
    b.check_in,
    b.check_out,
    b.guests_count,
    b.total_amount,
    b.status,
    p.base_price,
    p.max_guests,
    p.property_latitude AS latitude,
    p.property_longitude AS longitude,
    d.destination AS city
FROM bronze.bookings b
```

```

JOIN bronze.properties p ON b.property_id = p.property_id
JOIN bronze.destinations d ON p.destination_id = d.destination_id
WHERE d.country = 'Japan'
      AND YEAR(b.check_in) = 2024
      AND b.status IN ('confirmed', 'completed')
      AND b.total_amount IS NOT NULL
      AND b.check_in IS NOT NULL
      AND b.check_out IS NOT NULL;

```

3.4 Key Transformations — Gold Layer (SQL)

```

%sql
CREATE SCHEMA IF NOT EXISTS gold;

CREATE OR REPLACE TABLE gold.monthly_price_trends AS
SELECT
    city,
    YEAR(check_in) AS year,
    MONTH(check_in) AS month,
    ROUND(AVG(total_amount / NULLIF(DATEDIFF(check_out, check_in), 0)), 2) AS
avg_price,
    ROUND(MIN(total_amount / NULLIF(DATEDIFF(check_out, check_in), 0)), 2) AS
min_price,
    ROUND(MAX(total_amount / NULLIF(DATEDIFF(check_out, check_in), 0)), 2) AS
max_price,
    COUNT(*) AS booking_count,
    SUM(DATEDIFF(check_out, check_in)) AS total_nights
FROM silver.japan_bookings_2024
GROUP BY city, YEAR(check_in), MONTH(check_in)
ORDER BY city, year, month;

CREATE OR REPLACE TABLE gold.geo_price_index AS
SELECT
    property_id,
    city,
    YEAR(check_in) AS year,
    MONTH(check_in) AS month,
    ROUND(AVG(total_amount / NULLIF(DATEDIFF(check_out, check_in), 0)), 2) AS
avg_price,
    ROUND(MIN(total_amount / NULLIF(DATEDIFF(check_out, check_in), 0)), 2) AS
min_price,
    ROUND(MAX(total_amount / NULLIF(DATEDIFF(check_out, check_in), 0)), 2) AS
max_price,
    latitude,
    longitude
FROM silver.japan_bookings_2024
GROUP BY property_id, city, YEAR(check_in), MONTH(check_in), latitude, longitude
ORDER BY city, year, month;

```

4. Interactive Dashboard

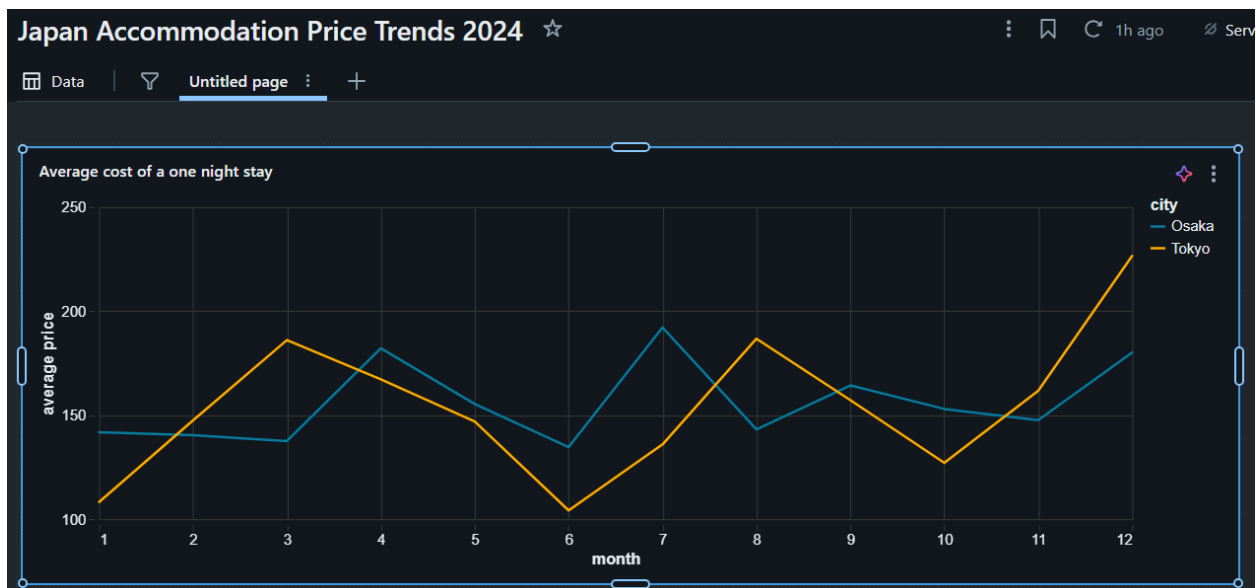
4.1 Target Persona

The dashboard can be used by to different types of clients:

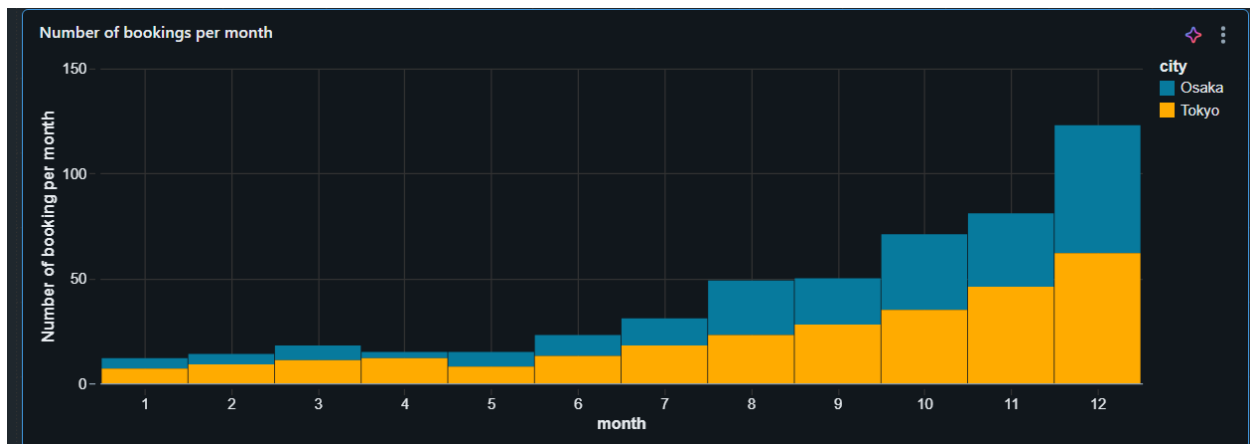
- Renters / Hosts
 - want to see when prices peak so they can maximise revenue. They need month-by-month avg_price and booking_count to understand yield opportunities.
- Tourist Agencies / Travellers
 - want to identify the lowest-cost windows to book trips. They need the combined city view to find months where both Tokyo and Osaka are affordable.

4.2 Charts

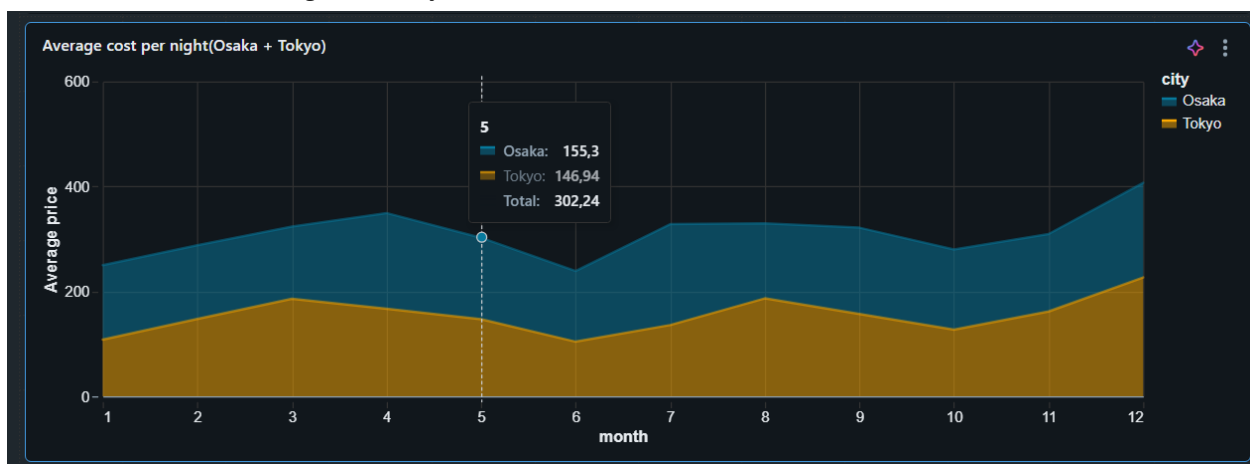
Chart	Description
Chart 1: Avg Price by Month	Line chart — X: month name (Jan–Dec), Y: average nightly price (¥). Two series: Tokyo (blue) and Osaka (orange). Allows direct city comparison across the year. Identifies peak months (March, April and December) where prices surge, and low-season troughs (January).
Chart 2: Booking Volume by Month	Grouped bar chart — X: month, Y: booking_count. Two bars per month (Tokyo, Osaka). Shows demand seasonality independent of price.
Chart 3: Combined Monthly Spend (Tokyo + Osaka)	Area chart — X: month, Y: sum of average prices for both cities combined. Represents total cost if a traveller visits both cities in the same month. Show when is the cheapest to book if you are visiting both cities.



Picture 1. – Chart 1: Avg Price by Month



Picture 2. - Chart 2: Booking Volume by Month



Picture 3. – Chart 3: Combined Monthly Spend (Tokyo + Osaka)

4.3 Stakeholder Narrative

The dashboard header includes a short written narrative for non-technical stakeholders:

"This dashboard tracks monthly accommodation prices across Tokyo and Osaka for 2024, based on confirmed booking data. Prices tend to peak during spring (March–April, cherry blossom season) and at the end of the year. The most affordable months for travel are January and June. Tourist agencies looking to organise budget-friendly trips to both cities should target these off-peak windows, where combined nightly rates are lowest. Hosts and property owners can use the same data to set premium pricing in high-demand months and lower prices in the quieter periods."

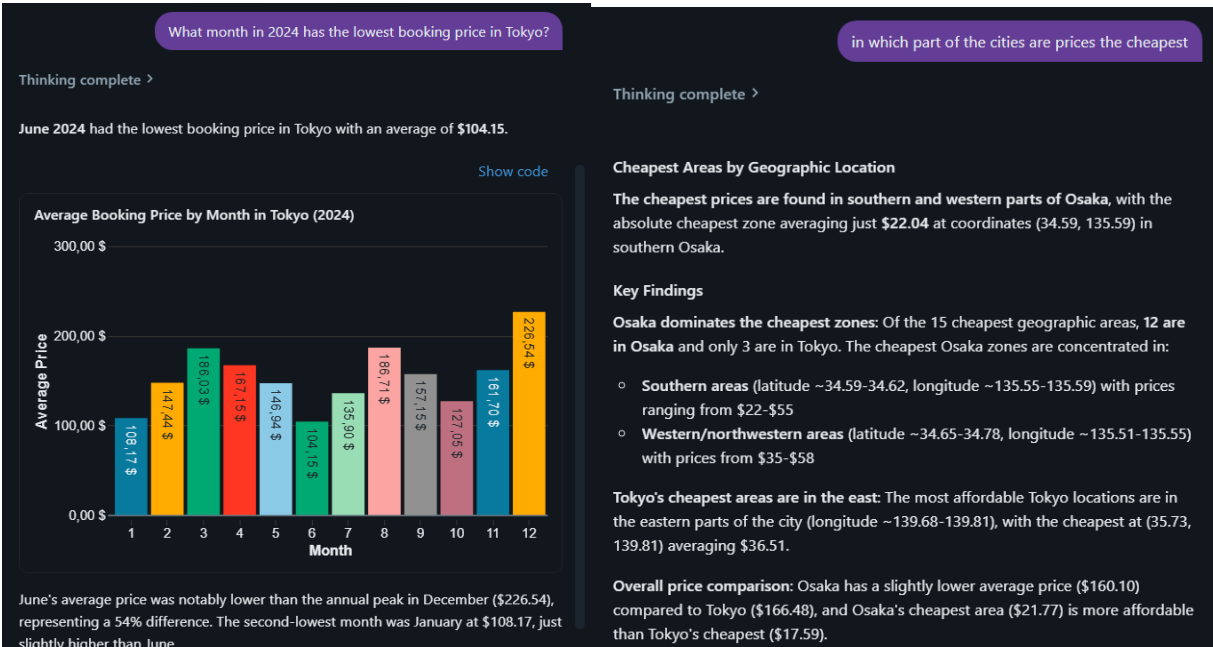
5. AI / Natural-Language Interface

5.1 Implementation

A Genie Space is configured over gold.monthly_price_trends and gold.geo_price_index. Genie Space is Databricks' native text-to-SQL product, requiring no custom LLM code — the user types a natural-language question and Genie generates and executes the SQL, returning the result.

5.2 Example Queries & Expected Outputs

Question	Answer
Q: What month in 2024 has the lowest booking price in Tokyo?	A: June 2024 has the lowest booking price in Tokyo with an average of \$104.15.
Q: In which part of the cities are prices the cheapest?	The cheapest prices are found in southern and western parts of Osaka with the absolute cheapest zone averaging just \$22.04. Tokyo's cheapest areas are in the east, averaging \$36.51



Picture 4. and 5. – Queries for the Genie Agent and it's answers

6. Write-Up & Trade-Off Discussion

6.1 What Was Built

- Bronze layer: Raw Delta tables read from Wanderbricks with no modifications.
- Silver layer: A single enriched, filtered, deduplicated table (silver.japan_bookings_2024) scoped to confirmed bookings in Tokyo and Osaka in 2024.
- Gold layer: Two analytical aggregates — monthly_price_trends (for the dashboard) and geo_price_index.
- Dashboard: Three charts in a Databricks SQL dashboard with a written stakeholder narrative.
- AI layer: Genie Space with 2 demonstrated natural-language queries.

6.2 Trade-Offs Made

Decision	Rationale & Trade-off
Scope to 2024 only in Silver, not Gold	Filtering early reduces the data volume processed at every Gold aggregation run, improving performance. Trade-off: the pipeline would need modification to serve multi-year analysis.
Price calculated as total_amount / nights, not base_price	total_amount is the actual transaction value (includes fees, demand pricing), making it more meaningful for trend analysis than the static base_price. Trade-off: it conflates length-of-stay effects with price changes.
Outlier removal using IQR method	Removes extreme values (e.g. luxury villas or data errors) that would skew monthly averages. Trade-off: may exclude legitimate premium listings. Mitigation: geo_price_index preserves property-level data for deeper investigation.
Genie Space over custom LLM code	Genie is native, requires zero maintenance, and handles SQL generation transparently. Trade-off: less customizable. Acceptable for a text-to-SQL use case over structured gold tables.

6.3 What I Would Do Differently With More Time

- Add a geographic map chart (Databricks Map visualization or Folium in a notebook) to geo_price_index, letting users click on neighbourhoods within Tokyo/Osaka to see price variation.
- Extend to 2023–2025 to enable YoY comparison (post-COVID recovery vs. 2025 peak season trends).
- Add a representation of price for each size of accommodation and have it mapped on a map on the dashboard

7. Dataset Justification

The Wanderbricks dataset was selected for the following reasons:

- Direct fit: It is a hospitality/booking dataset, making it a good choice for accommodation price trend analysis.
- Sufficient: The three core tables (bookings, properties, destinations) provide all necessary dimensions, price, location, date and property type, to get results that the clients need.
- Reproducibility: It is available natively in Databricks Community Edition. Any reviewer can clone the repo and run the notebooks immediately.
- Realistic data volume: The dataset is large enough to produce meaningful insights (monthly trends, city comparisons) while remaining manageable on a single Community Edition cluster without timeout risk.